TEAM PROJECT PART 2

LAB GROUP/TEAM:	P4/4
STUDENT NAMES & ID:	ONG HONG LIANG (2301859) HENG SENG HONG, ROYSTON (2301799) MUHAMMAD AZZI IZZUAN BIN AZAHAR (2301955) CHEAH ZHENG FENG (2302623) FOO ZHAN YONG (2301905)

Overall System Design

Overview of Components and Layers used

The overall system design of our game is split into two different layers, the game layer and the game engine layer. The game layer contains components that dictate the game's specific rules, behaviors and features. On the other hand, the game engine layer provides a number of generic, reusable components and services that supports the game layer, dispensing key functions such as rendering, physics and input handling.

Game Engine Layer

The game engine layer offers general-purpose components that form the backbone of multiple games.

Entities Component

In the game engine layer, the **Entity** component is a key component that plays a role as a template for all game entities. Using textures and sprites, it encapsulates common attributes like position (x, y coordinates), speed, and graphical representations, creating a consistent framework for rendering and interacting with these objects in the game world. With a physics body and unique identifier assigned to each entity, the game's physics engine can simulate complex physical interactions and behaviors. The interaction mechanics of the game, like movement and collision handling, are supported by this system in addition to making the visual representation and animation of game elements easier.

Base Factory Component

The BaseFactory component is an essential game engine component that generalizes object creation across the structure. The factory design pattern, which is in charge of producing different parts and objects in the game, is encapsulated by it, as it offers a common interface and shared functionalities for all kinds of factories. Moreover, the GameMaster is the main orchestrator of the runtime environment of your game, and the BaseFactory contains a reference to it. This design provides access to the GameMaster for any derived factory class, allowing them to seamlessly create or oversee game entities, UI elements, or system managers in a manner that aligns with the overall game environment. A more manageable and scalable system design is made possible by centralizing the game reference in the BaseFactory, which also makes the architecture simpler and less redundant. New factory types can be added to the system with little to no disruption to the current infrastructure.

Managers Components

The manager components are a group of abstract elements in the engine layer that individually specify the functionalities for managing various game elements. All managers in the game engine use the **BaseManager**

component as a standard template, outlining the essential functions that every manager must carry out. The **EngineCollisionManager** component describes the interface for handling collision detection and response logic, while the **EngineAIControlManager** component describes the features required to control AI behavior within the engine. The lifecycle and state management framework for entities is provided by the **EngineEntityManager** component. The game layer will extend the core input/output processes established by the **EngineInputOutputManager** component to handle specific input and render output. The fundamental functions for organizing game maps and associated content are laid out by the **EngineMapManager** component. The **EnginePlayerControlManager** component explains the fundamental player entity control mechanisms, while the **EnginePhysicsManager** component specifies the specifications needed to perform physics simulation and interaction. The **EngineSimulationLifeCycleManager** components describe the lifecycle events within the simulation, such as initialization, pausing, and resuming, while the **EngineSceneManager** components set up the template for scene transitions and management. Lastly, the **EngineSoundManager** component designates the operations for handling the audio playback and sound effects of the game.

Utility Components

Resource management and development process optimization are greatly aided by the game engine's utility modules, such as **TextureLoader** and **FactoryProducer**. The **FactoryProducer** component is a hub for dynamic creation that uses reflection to instantiate different types of factories, depending on the needs and contexts of the game, such as factories for creating managers, entities, or UI components. This makes it possible to generate game components in a flexible and scalable manner, making it simple to expand and customize the game engine's functionality. Following this, the **TextureLoader** component plays a role as an integrated resource manager for graphical assets, loading and caching textures efficiently to prevent excessive loads and refine memory usage. It makes sure that the right assets are used to visually represent game entities by intelligently searching through predefined directories to locate and load the requested textures. It also offers a way to remove textures when they are no longer required, which improves the engine's resource management and performance even more. In the architecture of the game engine, these two utility components serve as crucial building blocks that facilitate the creation of rich and engaging gaming environments while streamlining asset management and component generation.

Game Layer

The game layer is dedicated to implementing the game's distinctive logic, interactions, and player experience.

AI Control Component

In the game system, it integrates a methodical approach to handle the movement of AI entities, streamlined by a series of unique classes each customized to unique movement styles. The **Movement** component extends from the AIController and sets up the key logic for AI movement providing versatility for basic movements such as moving left or moving right, as well as incorporating Box2D physics for more intricate nuanced motion control.

By emphasizing horizontal traversal, allowing entities to move left or right within the game environment, and adding randomness to movement to add unpredictability to AI behavior. Overall, the AI control enhances the complexity of the game and the interaction between the player entity and the AI entity by adding obstacles and randomness to the game.

CollisionHandler Component

The collision handling system of the game is controlled by a hierarchy of collision handlers, each designed to handle a particular kind of interaction in the game environment. The implementation of a method for handling collisions between any two objects is required by the **CollisionHandler** component, which serves as the basic template for all collision handlers. For the **DefaultCollisionHandler** component, it handles collisions in a generic manner, not altering the game state or causing any special effects beyond logging the collision's occurrence. The **PlayerAICollisionHandler** component is responsible for handling collisions between the player and AI entities. These collisions usually have an impact on the player's lives and produce a collision sound. For the **PlayerItemCollisionHandler** component, it handles collisions between the player and objects. It can be used to apply an item's special effects on the player, remove the object from the game, or increase the player's jump count. Last but not least, the **PlayerPointsCollisionHandler** component handles collisions between the player and special point triggers, such as death or exit points. It can result in the player dying, the level finishing, or the player's jumping ability being reset.

Entities Components

In the game system, several entity components each serve a distinct function to enhance gameplay and interaction. For in-game objects that are not under the player's control, such as AI or items, the **NonPlayable** component serves as a designation. It offers a centralized method for classifying and controlling these objects by subtyping identification. The **Item** component is the core to the interactivity of the game, which represents the object's players can collect or interact with, and also impacts the game's dynamics by modifying the scores, player abilities or health when the player collides with it. For the **Player** component, it represents the player's character, with its unique attributes such as lives, score, speed and jump height, alongside with functions that adjust these attributes. For the **AI** component, it represents the computer-controlled entities that are equipped with capabilities like movement and awareness, which facilitates specific interactions with players or the game environments based on its predefined behaviors. Last but not least, the **itemEffects** component uses the itemEffectsMapper to coordinate the application of different effects on the player that are brought about by item collection. This is achieved by using a structured mapping between item types and their corresponding impact functions, which gives item interactions additional layers of strategic depth and consequence.

Factory Components

The game layer consists of diverse factory components, each designed to enhance specific facets of the game development and interaction dynamics. The **BodyFactory** component plays a key role in creating physical bodies for entities in the physics world, which allow the entities to take part in physics simulations. On the other

hand, the **CollisionHandlerFactory** component is essential to creating custom collision handlers. As these handlers are skilled at identifying the kinds of objects involved in collisions, they can organizes tailored responses that increase the complexity of the gameplay. The **EntityFactory** component serves as the creative hub for spawning entities in games. It provides the flexibility to dynamically generate a wide variety of entity types, each with distinct characteristics and functions within the ecosystem of the game. Following this, the **ManagerFactory** component plays a key role in the assembly of game manager instances, which are necessary for the organized and effective management of different game subsystems. Lastly, the **ScreenFactory** component is dedicated to creating the user interface. It does this by designing the screens that act as entry points between the various game sections, from the main menu to the play screen. This effortless navigation is crucial in ensuring an integrated and engaging gameplay.

Input/Output Components

The IOManager component, which extends the EngineIOManager from the game layer, is responsible for handling input/output (I/O). The IOManager's encapsulation enables an intricate and expandable I/O system that meets the complex requirements of the game's control system.

LevelConfigs Component

In the game layer, the game's level configuration components each play an essential role in constructing the game levels. The **LevelLoader** assists in loading configurations from JSON files, allowing agile level setups enhanced with various elements obtained from external data. The **LevelConfig** operates as the blueprint for each individual level, describing facets such as the map paths, items, random items, walls, and the required score to progress into the next round. There are subcomponents within the **LevelConfig**, which are the **Item**, **RandomItems** and **Wall** components.These components add complexity to the game's environment. The **Item** component describes the types and number of items in the level, offering a variety and strategy to the gameplay. The **RandomItems** component presents unpredictability and variety to the game by enumerating the number of random items to be generated within the level. Lastly, the **Wall** component details static boundaries and obstacles within the level, encompassing their type and coordinates to build the level's layout and challenges.

Managers Components

In the game layer, each manager has a niche component tasked to oversee a distinct component, essentially contributing to the integrated operation of the game environment as a whole. The **AIControlManager** is accountable for instructing the behavior and movement of AI entities, assimilating navigation and decision-making logics. The **CollisionManager** handles collision detection and responses, prompting for appropriate collision handlers based on the entities involved. The **EntityManager** handles the lifecycle of all game entities, such as their creation, updating and deletion. The **MapManager** handles all operations of the game maps, such as loading data, rendering, and managing map-specific entities such as the spawn points of the AI entities and Player entity. The **PhysicsManager** handles the simulations of the game's physics, affecting the movement of the entities, collision physics and the overall physics world. The main focus of the

PlayerControlManager is the interactions and actions of the players, such as the movement and the jump mechanics. The **SceneManager** manages the transitions between the game screens, such as transitioning the main menu to the gameplay screen, and the gameplay screen to the win/lose screen. The **SimulationLifeCycleManager** handles the game's simulation lifecycle, such as the level setup, entity resets and the transitions of the game state. Lastly, the **SoundManager** is accountable for all the sound and music playback, uplifting the game with background music and sound effects in the gameplay.

Screens Components

In the game layer, screen components play an essential role in refining player interaction and the game flow. The **BaseScreen** plays a role as an abstract core of the screen components, where each screen component is an instance of a class that is produced from the BaseScreen. This makes it consistent while it requires unique modification for each part of the game's flow. The segmentation of the screen components allows smooth updates and additions to the game's essential UI construction and interaction functionalities. The **MainScreen** plays a role as a Main Menu interface for the players, giving the players options like starting a new game or leaving the game. The **PauseScreen** allows players to take a break during the game, and also they are allowed to modify the volume or go back to the main menu (MainScreen). The **PlayScreen** is where the game action is being played, where it manages the game world's rendering, player inputs and the game state updates. The **ScreenTextureObject** streamlines textured object management within the screens, mitigating image setup and manipulation. Lastly, the **WinLoseScreen** ends the game by displaying the game results, either a win or a loss, and offers options to play the next level if the player completes the level, or return to the main menu, or exit the game, which creates an extensive and intuitive game experience.

Design Patterns Implemented

Lazy Initialization in GameMaster: Lazy initialization has been implemented in the GameMaster class through the getManager method, which optimizes resource management and enhances the performance. Upon its first use, the GameMaster class produces and stores each manager rather than instantiating them all at once. This approach efficiently uses resources by avoiding the upfront allocation of objects that may not be immediately needed, checking for an existing instance within a list before creating a new one.

Factory Pattern Implementation

The Factory pattern is another implementation from the GameMaster class. The getManager and getFactory methods are used to implement it, giving object creation and management an organized structure. If a new manager or factory is needed and does not already exist, the GameMaster delegates the creation of the new manager or factory to the appropriate factory class, either FactoryProducer or ManagerFactory. In doing so, it abstracts the instantiation logic from the consumer and encapsulates object generation. Due to the ability to choose the manager or factory type to be built at the execution time, this offers flexibility and decoupling.

Singleton Pattern in Game Screens and Managers

Usage of the Singleton pattern can be seen in the GameMaster class via the management of the game screens and managers. By initializing only one of each manager and screen, the resources required are lessened while ensuring we still maintain the necessary access point to the relevant managers. Using the getManager method will only reference an existing manager if it is present or create one if it has not been done before, thereby displaying the Singleton Pattern.

Command Pattern Utilization/ Strategy Pattern Adaptation in ItemEffectMapper

Command pattern implementation can be seen in the itemEffectMapper. It encapsulates the effects of items on the Player into a lambda expression in the Consumer<Player> interface. It will then send a request to the Player object to execute the commands which will in turn cause the Player to react a certain way in response to the effects. The itemEffectMapper also operates the **strategy pattern** concurrently. Since there are a variety of items to interact with the Player, different effects corresponding to the items will be applied to the Player. Depending on the item, the applyEffect method will dynamically select the appropriate strategy and the effects will be applied to the Player from the effectsMap. By only choosing the correct effects to apply, this showcases the effectiveness and flexibility of the Strategy Pattern as it can make changes to the item effects without modifying the Player class or the classes that support the Player.

Abstract Factory Pattern in Engine Layer

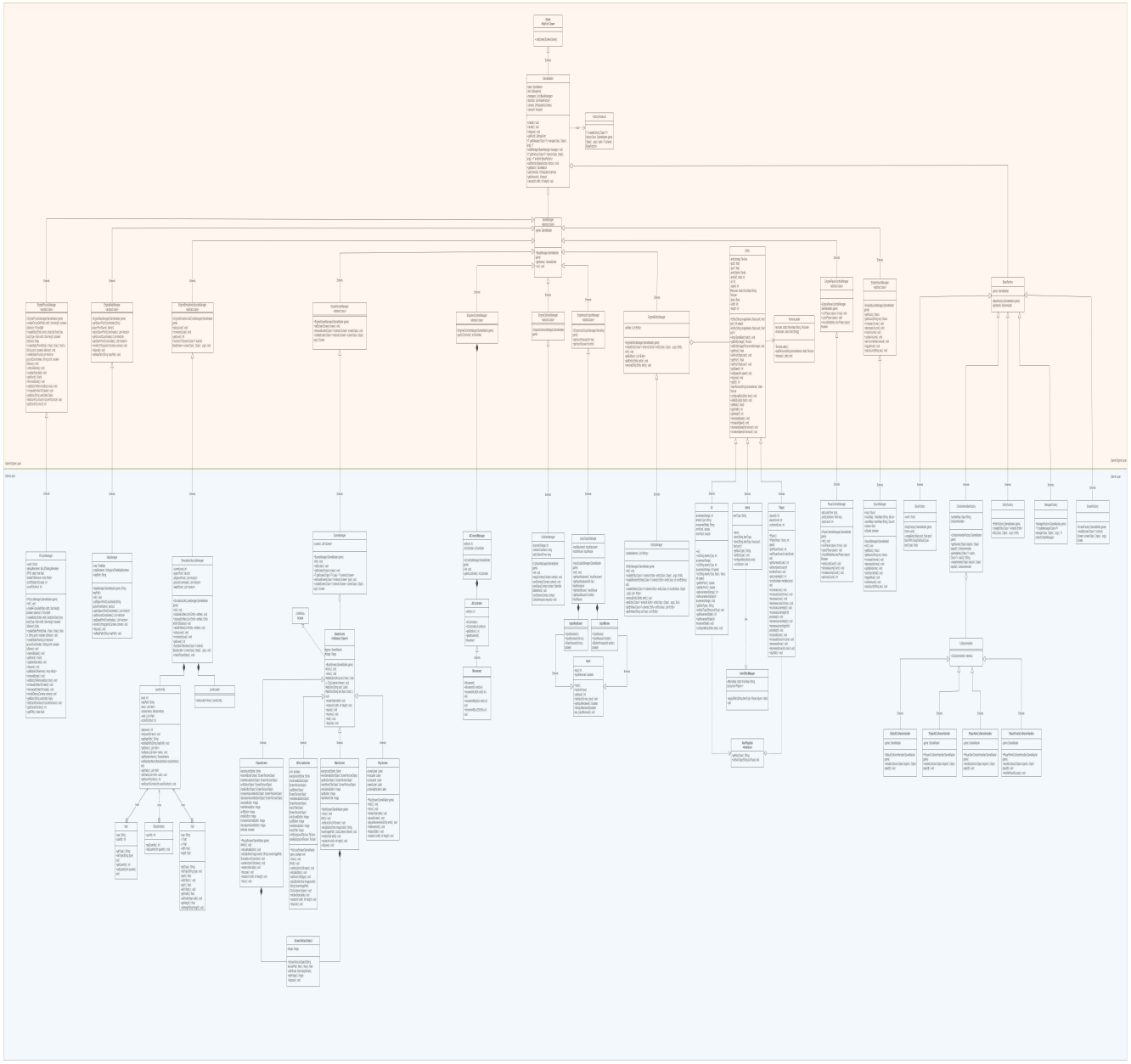
One component of the game engine that complies to the Abstract Factory pattern is the BaseFactory. It serves as an abstract layer that sets a standard for building families of linked or dependent objects without defining their concrete classes. In this instance, the FactoryProducer is in charge of returning these concrete factory objects, and BaseFactory offers a standard interface for factories. The structure that enables expanding BaseFactory to construct custom factories for various uses, including entity creation, screen generation, or game management, clearly follows a pattern. By allowing the GameMaster class to create a group of linked objects without having to know the details of each one, it supports the idea of "programming to interfaces, not implementations."

Improvements to the engine from Last Submission

After receiving feedback regarding our game engine, our group has revamped our management systems and added factory implementations. These changes were crucial in creating an adaptable and maintainable framework for game development. Our original design for the game engine was simpler, with a manager class for overseeing each game layer feature. However, that comes with the issue of the managers being extremely limited, making it harder to customize. As such we decided to use abstract manager classes to increase the versatility of our architecture. After updating the code with this, the overall game engine has become more flexible for our team.

BaseFactory works together with this updated engine. It is a core component that is designed to be used as a pipeline in regards to the object creation process in the game engine. Factories in the game layer can extend from this base and make use of the methods while having the flexibility to implement game-specific object constructions. This structure shows our desire to ensure our code is reusable and can be extended repeatedly based on what we learned in Object Oriented Programming. After remaking our engine's foundational structure, it now supports developments of games more effectively and with more creativity. Our game engine has been rebuilt from a rigid framework to one that provides aid to developers so that they can innovate and create more entertaining games.

UML Diagram



Object Oriented Principles Adopted

Encapsulation

One of the key principles of object oriented programming, encapsulation is the hiding of data. Only the behavior of the data is accessible by the outside while the data itself is hidden and protected. This is done by controlling the level accessible modifiers. By doing so, we achieve maintainability, security and extensibility. Maintainability is achieved as if encapsulation is properly implemented, the one in charge of the code will be able to understand how to use it without knowing in detail how it works. Security is established as encapsulation restricts the access to data and provides a layer of security to the code. Finally, extensibility is present because code that has related data and behaviors are grouped together. Any changes can be done to the encapsulated part without affecting the input or output formats. Examples of usage of encapsulation are the "private" modifiers for the attributes in our code, and to access the data, we use getters and setters.

Inheritance

Inheritance is another fundamental concept of object oriented programming. It allows a subclass to reuse properties and behaviors of its parent class. This promotes creation of hierarchical structure. It is essential in code maintainability and scalability. By using inheritance properly, developers will be able to understand the relationship the classes have with one another as well as the properties they possess. Scalability is achieved as additional subclasses of the superclass can be added without modifying the pre-existing subclasses. An example of inheritance is how all the game related screens extend from BaseScreen. As BaseScreen contains the attributes required for a screen, while the screens such as PlayScreen and PauseScreen have their own specializations with their own attributes necessary for their functions.

Polymorphism

Polymorphism is the third principle of object oriented programming and it involves allowing objects to take on different forms or exhibit different behaviors. This is done through method overloading and method overriding. Polymorphism helps with code reusability, extensibility and it promotes loose coupling. Polymorphism allows the code to be written in a broad way that many objects of different types can use. Extensibility and reduced coupling are brought out using interfaces as more classes can be added with specific methods without conflicts or modifications of previous classes. An example of polymorphism is the dispose() method. The name "dispose" is shared among different classes, from the Screen to Entity classes. But they have similar functions and are used to remove things when they are no longer being used.

Abstraction

Abstraction is the final principle of object oriented programming which is hiding complex, low-level details and only showing the essential information to the user. By only showing the essential features of the systems and

leaving out the irrelevant information, developers can focus on the core aspects that are important to the problem currently and they will be able to interact with the engine without knowing how the data is stored or used. The engine managers are examples of abstraction. They serve as the blueprint to the managers in the game layer. The managers of the game layer extend from the engine managers and use the abstract methods in order to fulfill their roles. This promotes scalability as a new abstract manager could be added without interfering with pre-existing managers.

Dynamic Item Creation

Automated Asset Recognition

The game engine has an inbuilt function that will access the necessary folder and will sort through which files can be created as an Entity. New images can be added and it will be checked which shows scalability. Those that can be generated, will be added into a list.

Robust and Adaptable Design

The game is robust as it ensures that there will be a default set of items that will be used in the case that the necessary directory goes missing. This backup is so there is no single point of failure if the assets are no longer available.

Dynamic Spawning Logic

Due to how the game is implemented, the items that are created based on the generated list as well as their locations will be randomly chosen to add unpredictability and variation to the game.

Strategic Advantages

Firstly, it enhances the game's extensibility, allowing for the effortless addition of new items and features. Secondly, it increases the game's flexibility, supporting rapid content iteration and expansion. Lastly, it streamlines the content update process, significantly reducing development time and effort.

Conclusion

To sum it up, dynamic item creation is proof of the engine’s progressive design. It strengthens developers and designers alike to add scalability into the game without hurting the rest of the game. It is using modern development practices to supervise creative expression and growth of the game’s future.

Level Configuration via JSON

Innovative Approach

We've introduced a transformative approach to level design within our game engine by utilizing JSON for stage configuration. This method significantly reduces the process of designing and going over different levels of design layouts.

JSON Configuration Mechanics

The game engine uses JSON files to initialize the static configurations of a particular level, such as the game obstacles, and specific gameplay elements like items and random objects if needed in the level. Each JSON file corresponds to a level, outlining parameters like the map's path, items with their types and quantities, walls, and scoring conditions needed to progress. With this map path, we are then able to dynamically retrieve the important game specific parameters such as the spawn points of players and enemies and the fall off point which varies depending on the map.

Advantages of JSON in Level Design

Design Efficiency: Level designers can quickly test out game designs and gameplay mechanics by editing the JSON file without the need for deep dives into the codebase.
Flexibility: The ability to define and modify game levels through JSON files enables a dynamic approach to game development, to assist in prototyping as well as testing.

Application in Game Engine

In the SimulationLifeCycleManager, the setupLevel function instantiates the game entities and structures based on the JSON configuration by parsing the level-specific JSON file to set up the game map, spawn points for the different entities, and static objects like walls and death points. This abstracts the level setup process from the game itself. As such, it focuses on setting up the things instead of DECIDING what entities to set up, which brings us back to single responsibility(SRP).

Reflecting on Practical Benefits

With the implementation of JSON, we can always set up a new level by creating a new json level file for the game easily, instead of having to look into the code and then having to design a whole new staging area for that specific level, which definitely violates the spirit of Object-Oriented Programming.

Reflection-Driven Object Instantiation Strategy

Our game engine embraces a reflection-driven approach for object instantiation that enhances its modular architecture and facilitates ease of expansion.

Reflection Across Engine Components

Reflection is employed throughout various components of the engine, such as the ManagerFactory and ScreenFactory. By utilizing Java's reflection API, these factories dynamically instantiate objects and invoke the appropriate constructors for any given class type.

ManagerFactory and ScreenFactory

By using reflection to create manager instances, the ManagerFactory greatly reduces the need for multiple, separate instantiation functions and reuses the logic for manager creation. Similar to this, the ScreenFactory creates instances of Screen objects dynamically by utilizing these concepts. This standardized approach streamlines the development of these fundamental engine parts, increasing its efficiency and adaptability.

Benefits of Using Reflection

1. Scalability: The engine can accept new types of managers and screens without the need to rewrite or add additional factory methods.
2. Flexibility: By abstracting the creation process, we provide a means to instantiate various objects at runtime based on the game's changing requirements.
3. Maintainability: Reducing the repeatable chunks of code associated with object creation helps maintain a clean and maintainable codebase, easing future development and refactoring.
4. Implementing a Cohesive Creation Pattern: By combining object creation into a reflection-based pattern, we make the engine's overall approach understandable and uniform. It illustrates our dedication to using cutting-edge programming methods to expedite the development process.

Conclusion

Reflection-based instantiation represents a strategic decision to future-proof our game engine. It signifies our pursuit of an advanced but straightforward solution to object creation. It is evidence of our dedication to innovation and excellence in game engine development.

Limitations with Engine, Game or Software Design

There are hardcoded values which could have been settled using a method. However due to time constraints, we were unable to implement it. There is a lack of error handling for most methods and those that do usually only print a message in the console instead of solving them.

Workload Distribution

Team member	Contributions	Shared Contributions
Zheng Feng	GAME ENGINE : **Engine Managers:** CollisionManager,MapManager,PhysicsManager, PlayerControlManager,SceneManager, SimulationLifeCycleManager, SoundManager **Factory:** BaseFactory **Entities:** EngineEntity GAME LAYER : **CollisionHandlers:** CollisionHanderInterface, DefaultCollision, PlayerAi,PlayerItem,PlayerPoints **Entities:** itemEffectMapper, AI, item, Player **Factories:** BodyFactory, CollisionHandlerFactory, Entity Factory ,Manager Factory,Screen Factory **LevelConfigs:** LevelConfig + LevelLoader **Game Managers:** CollisionManager, EntityManager, MapManager, PhysicsManager, PlayerControlManager, SimulationLifeCycleManager	VIDEO REPORT SLIDES
Zhan Yong	GAME ENGINE : PlayerControlMovement, UML GAME LAYER : PlayerControlMovement, UML, TiledMap Design, UI Testing	
Hong Liang	GAME ENGINE : CollisionManager, UML GAME LAYER : CollisionManager, TiledMap Design, UI Testing, PlayScreen, UML	
Azzi	GAME ENGINE : SimulationLifeCycleManager, SoundManager GAME LAYER : **Managers:** SimulationLifeCycleManager, SoundManager **Screens:** MainScreen, PauseScreen, PlayScreen, WinLoseScreen, ScreenTextureObject, TiledMap Design, UI Testing	

Royston	GAME ENGINE : AIControlManager, InputOutputManager GAME LAYER : **Managers:** AIControlManager, InputOutputManager **AIControl:** AIController, Movement **InputOutput:** Input, InputKeyboard, InputMouse	